Failure (accidentally blocking the event loop) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `blocking_event_loop_practice.py`: `time.sleep()` inside `async def` doesn't error, doesn't warn, and still gives a correct final answer — the only symptom is that everything took twice as long. **If not:** the first time you'd meet this bug would be in real code, as an unexplained slowdown with no obvious cause, instead of something you caused on purpose here and can now recognize immediately.

Solution

```
# blocking_event_loop_practice.py
import asyncio
import time

async def good_task() -> str:
    await asyncio.sleep(3)
    return "good"

async def blocking_task() -> str:
    time.sleep(3)             # the bug: no await, freezes the whole loop
    return "blocking"

async def time_it(label: str, coro_a, coro_b) -> None:
    start = time.perf_counter()
    results = await asyncio.gather(coro_a, coro_b)
    elapsed = time.perf_counter() - start
    print(f"{label} took {elapsed:.2f}s -> {results}")

async def main() -> None:
    await time_it("two good tasks:", good_task(), good_task())
    await time_it("blocking + good:", blocking_task(), good_task())

if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
two good tasks: took 3.01s -> ['good', 'good']
blocking + good: took 6.02s -> ['blocking', 'good']
```